AF SECTIONS 1 TO 10

LAYER 1, AUTHORITY AND DOCTRINE

Internal Use Only. Not for external distribution.
Version: L1-v1 — Human authority reference for intake feasibility gating.

How to use this document

This is the first-read authority document for AF intake. It explains what intake is, what it is not, what is locked, why those locks exist, how intake reaches a determination, and what must never be disclosed externally.
Use this document to answer:

- what intake is supposed to do
- what intake is not allowed to become
- what intake outputs mean
- what internal nondisclosure discipline applies
- what external vocabulary is mandatory or prohibited
- how run sufficiency, stop rules, and sequencing work
- how complaint structure is preserved without allowing grouped drafting to control execution atomics
- how contexts, notes, and constraints are governed
  Use Layer 2 when the question is, "What exactly must Jules build, validate, output, and prove for intake?"
  Use Layer 3 when the question is, "What are the exact fields, enums, templates, conditionals,

context locks, sequencing rules, and machine-bindable constraints?”

SECTION 1: PURPOSE AND BOUNDARY OF INTAKE [LOCKED]

1.1 Intake as a bounded feasibility gate

Intake is a bounded feasibility gate. It determines whether the matter qualifies for full forensic execution under controlled browser contexts, using only the complaint or demand materials provided as the scope anchor.

1.2 What intake is not

Intake is not:

- a compliance audit
- a defect inventory
- a remediation guide
- a certification
- a legal opinion

1.3 Intake output boundary

Intake outputs eligibility only. Intake does not disclose internal depth, internal selection detail, counts, run volume, or internal sequencing.

1.4 Locked nondisclosure posture, external outputs

Any external-facing intake output must not disclose:

- number of runs performed
- number of confirmations reached
- any observed or not observed counts
- cap reached status
- which specific asserted conditions were selected as run units during intake
- per-run sequencing details
- per-run or per-asserted-condition context assignment beyond the matter-level context disclosed by the determination template or other governed external intake template

Matter-level execution context may be disclosed using the locked exact terms Replicated Desktop Browser Context and Replicated Mobile Browser Context, including whether the matter was assessed under Desktop only, Desktop and Mobile, Desktop with Mobile baseline constrained, Mobile only, or Mobile with Desktop baseline constrained.

Such disclosure must remain at the matter level only. It must not disclose per-run sequencing, per-asserted-condition context assignment, or other internal execution structure.

1.5 Indirect signaling prohibition

External-facing intake outputs must not imply any of the prohibited internal details indirectly, including through phrasing such as:

- “extensive testing”
- “limited testing”
- “we checked everything”
- “we checked only a few items”
- or similar statements that signal internal execution depth

1.6 Intake is not a disguised merits process

Intake is a feasibility gate only. It does not become a disguised assessment, a partial audit, a remediation posture, or an external narrative about site quality, claim strength, or party conduct.

SECTION 2: AUTHORITY, SOURCE OF TRUTH, AND DRIFT CONTROL [LOCKED]

2.1 AFDM as the implemented intake system

AFDM is the implemented automated execution system that performs intake runs and emits intake artifacts. “AFDM repository behavior” means the actual outputs, labels, record fields, parameter locks, and constraint handling produced by the current implemented system. AFDM intake record schemas, field contracts, and

validation rules are defined in Layer 3 and must not be narrowed, expanded, or paraphrased at this layer.

2.2 AFDM repository behavior is the source of truth

AFDM repository behavior is the source of truth. This doctrine must not claim artifacts, parameters, viewport locks, label sets, stop rules, or behaviors that are not supported by the implemented system.

2.3 Locked discipline, internal records only

Intake records must not contain:

- evaluative fields
- intent signals
- strength indicators
- probability signals
- outcome-leaning metadata

2.4 Outcome categories must remain neutral

Outcome categories and record fields must not imply:

- compliance posture
- liability posture
- likelihood posture

2.5 Record terminology discipline

Record terminology must not frame entries as:

- findings
- violations
- assessments

2.6 Scope note

This section governs internal record structure and metadata discipline. External-facing vocabulary restrictions are governed by Section 3.

2.7 Conflict rule

If doctrine text conflicts with implemented AFDM behavior, runtime AFDM behavior governs what the system actually did. This does not authorize silent drift. Any identified conflict between doctrine text and implemented behavior must be surfaced and corrected through governance; operators must not resolve conflicts by improvising new fields, labels, or procedures.

2.8 Drift control

Any identified conflict or ambiguity between doctrine text and implemented behavior must be corrected in the doctrine text before the next release of the intake runner. Operators must not resolve conflicts by improvising new fields, labels, or procedures.

2.9 No improvisation rule

Operators must not cure doctrinal ambiguity by inventing:

- new metadata
- new record fields
- new labels
- new stop logic
- new scope logic
- new context logic
- new procedures

If the implementation and doctrine diverge,

governance must correct the doctrine. Operators do not get to patch doctrine by practice.

2.10 Why this section is hard-edged

This section exists to stop doctrine from drifting into fiction and to stop operations from drifting into improvisation. If AFDM does not support it, doctrine must not pretend it does. If AFDM behaves differently, the doctrine must be corrected through governance, not operator creativity.

SECTION 3: MECHANICAL OBSERVER LANGUAGE [LOCKED]

3.1 Mechanical neutrality

Intake language is mechanically neutral. It records observed state under bounded execution parameters. It does not characterize, evaluate, assign responsibility, imply motive, or imply intent.

3.2 Locked banned framing, principle-governed

The following terms are prohibited in all external-facing intake output. In addition, any functional equivalent is prohibited, even if not listed here:

- pass
- fail
- compliant
- non-compliant
- violation
- audit, as intake purpose
- remediation, as intake purpose
- certification

- guarantee
- any blame posture
- any adversarial posture

3.3 Definition of blame posture or adversarial posture

“Blame posture or adversarial posture” means any language that assigns fault, implies intent, suggests wrongdoing, or frames either party as acting improperly. Intake output must not contain such framing.

3.4 Governing principle

If a term characterizes the site, the claim, or either party, rather than recording a mechanically observed state under bounded parameters, it is prohibited whether or not it appears on the banned list.

3.5 Locked scope anchor terms, external outputs only, mandatory use

External-facing intake output must use the following exact terms when describing scope and execution context. Paraphrase is not permitted:

- “complaint or demand materials provided”
- “specific website conditions asserted in those materials”
- “bounded execution parameters”
- “Replicated Desktop Browser Context”
- “Replicated Mobile Browser Context”

3.6 Locked burden rule

Intake does not shift burden by rhetoric. Intake stays literal about:

- what was executed
- what was observed
- what was blocked by a constraint
- what could not be bounded due to missing specification

3.7 Why this section is strict

This section is not about style preferences. It exists to prevent intake from becoming argumentative, evaluative, or suggestive through wording. Intake records bounded execution state. It does not narrate fault, motive, strength, or legal posture.

SECTION 4: INTAKE DETERMINATION TEMPLATES [LOCKED]

4.1 Locked determination template set, eight options only

Intake issues exactly one determination per matter. External-facing intake output must use one, and only one, of the following fixed determination templates. No paraphrase, no alternate labels, and no additional templates are permitted.

The eight permitted determination templates are:

- DETERMINATION: ELIGIBLE FOR DESKTOP AND MOBILE TECHNICAL RECORD BUILD

- DETERMINATION: ELIGIBLE FOR DESKTOP TECHNICAL RECORD BUILD

- DETERMINATION: ELIGIBLE FOR DESKTOP TECHNICAL RECORD BUILD / MOBILE BASELINE: CONSTRAINED

- DETERMINATION: ELIGIBLE FOR MOBILE TECHNICAL RECORD BUILD
- DETERMINATION: ELIGIBLE FOR MOBILE TECHNICAL RECORD BUILD / DESKTOP BASELINE: CONSTRAINED
- DETERMINATION: NOT ELIGIBLE FOR FORENSIC EXECUTION
- DETERMINATION: NOT ELIGIBLE FOR FORENSIC EXECUTION - CONSTRAINTS (BOTMITIGATION)
- DETERMINATION: NOT ELIGIBLE FOR FORENSIC EXECUTION - CONSTRAINTS (OTHER)

These eight templates are the only permitted external determinations. Templates do not create additional determination categories beyond their locked meanings.

4.2 Locked nondisclosure, determination line

The determination line must not reveal, directly or indirectly:

- how close intake came to sufficiency
- which asserted conditions were selected or attempted
- what was observed or not observed
- whether any run cap or stop rule was reached

The determination line must not include modifiers or hedging, for example:

- provisionally
- appears
- strongly
- weakly
- likely
- unlikely
- or similar phrasing

4.3 Peer-baseline eligibility and constraint guardrails

Dual-baseline eligibility is expressed only through Template 1.

Desktop-only eligibility is expressed only through Template 2.

Desktop eligibility with a Mobile baseline constraint is expressed only through Template 3.

Mobile-only eligibility is expressed only through Template 4.

Mobile eligibility with a Desktop baseline constraint is expressed only through Template 5.

Template 3 is permitted only when Mobile baseline is in scope and Mobile baseline execution is blocked under controlled parameters, and the internal record uses the applicable constraint_class value.

Template 5 is permitted only when Desktop baseline is in scope and Desktop baseline execution is blocked under controlled parameters, and the internal record uses the applicable constraint_class value.

“Mobile-preferred,” “Desktop-preferred,” and “partial eligibility” determinations are prohibited.

4.4 Determination rigidity rule

The determination line is a locked output surface. It must not be expanded, softened, explained, or supplemented with informal gloss that signals internal depth or internal reasoning.

4.5 Why Section 4 is rigid

This is the most externally exposed intake output. If this section drifts, internal execution depth, sufficiency proximity, and internal path selection start leaking through wording even when the labels look controlled.

4.6 Constraint-driven ineligibility routing

DETERMINATION: NOT ELIGIBLE FOR FORENSIC EXECUTION is valid only when the matter is not eligible for forensic execution for reasons other than a constraint condition.

DETERMINATION: NOT ELIGIBLE FOR FORENSIC EXECUTION - CONSTRAINTS (BOTMITIGATION) is valid only when BOTMITIGATION is the controlling ineligibility basis.

DETERMINATION: NOT ELIGIBLE FOR FORENSIC EXECUTION - CONSTRAINTS (OTHER) is valid only when a locked constraint_class value other than BOTMITIGATION is the controlling ineligibility basis.

DETERMINATION: NOT ELIGIBLE FOR FORENSIC EXECUTION must not be used when any locked constraint_class is the controlling ineligibility basis. The BOTMITIGATION template must not be used for non-BOTMITIGATION ineligibility. The OTHER-constraints template must not be used when the controlling ineligibility basis is BOTMITIGATION.

SECTION 5: RUN CAP, SUFFICIENCY, AND STOP RULES [LOCKED]

5.1 Run cap

Maximum of 10 intake runs per matter. No time cap. No assertion-count cap. The run cap is the only bound on run volume.

5.2 Run definition

One run equals one controlled attempt of one asserted website condition under one browser context, producing exactly one outcome label from the locked outcome label set in Section 7.

5.3 Qualifying confirmations

The only outcomes that qualify as confirmations are:

- Observed as asserted
- Not observed as asserted
  Constrained and Insufficiently specified for bounded execution do not qualify as confirmations.

5.4 Sufficiency threshold

Sufficiency is reached when there are two qualifying confirmations, each produced by a distinct run, across any mix of contexts that are in scope for the matter. The two qualifying confirmations may come from runs executed against the same RUN_UNIT or against different RUN_UNITs; distinct runs are required, not distinct run units.

5.5 Stop rule

Stop immediately when sufficiency is reached, or when 10 runs are completed, whichever happens first. When sufficiency is reached on the same run that completes the 10-run cap, both stop conditions are met simultaneously; in that case, sufficiency is the stop basis for that matter.

5.6 No runs after stop

No additional runs are permitted after the stop condition is met for that matter, even if additional asserted conditions remain or new ideas arise.

5.7 Intake closure rule

A stop condition closes intake for that matter. Intake must not be resumed or extended without a new matter scope submission.

5.8 Cap Consumption Rule

Any intake run, regardless of outcome label, consumes one of the 10 permitted run slots for that matter. Runs labeled Constrained or Insufficiently specified for bounded execution count against the run cap but do not produce qualifying confirmations.

Sufficiency remains a mechanical threshold tied to run counts and locked outcome labels, not a narrative or confidence signal.

SECTION 6: COMPLAINT-ANCHORED NORMALIZATION [LOCKED]

6.1 Complaint structure preserved for traceability

Intake preserves complaint or demand structure for traceability, but normalizes execution into atomic run units so that grouped drafting does not control operational atomics.

6.2 Locked normalization ruleset, complaint group anchors

Each complaint or demand grouping is assigned a COMPLAINT_GROUP_ANCHOR using one of the following forms only:

- page_paragraph_range
- page_bullet_range

Operators must not invent alternative anchor types, for example:

- issue heading
- count
- subsection
- or similar constructs

without a doctrine update.

6.3 Complaint group anchor is a pointer only

A COMPLAINT_GROUP_ANCHOR is a reference pointer only. It does not control how many asserted conditions are executed.

6.4 Distinct asserted conditions become distinct run units

Each distinct asserted website condition within a COMPLAINT_GROUP_ANCHOR is assigned its own RUN_UNIT. A single COMPLAINT_GROUP_ANCHOR may produce multiple RUN_UNITs.

6.5 Independent execution requirement

Each RUN_UNIT is executed independently and yields exactly one outcome label from the locked outcome label set.

6.6 No blended testing

A RUN_UNIT must test exactly one asserted website condition, with no blended testing and no multi-assertion bundling.

6.7 Locked atomicity rule

RUN_UNIT is the only permitted execution atomic for intake. Each intake run must map to exactly one RUN_UNIT, and each RUN_UNIT must be executed by exactly one run under exactly one browser context.

6.8 Locked traceability rule

Complaint anchoring is preserved for reference integrity, but execution remains atomic at the RUN_UNIT level even when multiple assertions are grouped together in the submitted materials. Operators must not collapse multiple asserted conditions into a single RUN_UNIT for convenience or narrative symmetry.

6.9 Why this normalization exists

Complaint drafting may group assertions for pleading convenience. Intake preserves that grouping for traceability only. It does not let grouped drafting dictate execution atomicity.

6.10 Pre-execution normalization

All RUN_UNIT records for a matter must be created before any intake runs execute for that matter. No new RUN_UNIT may be created after the stop condition has been evaluated for that matter.

6.11 Unexecuted RUN_UNITs

The run cap may result in normalized RUN_UNIT records that are never executed before the stop condition fires. Unexecuted RUN_UNITs are not an error condition and must remain in the matter record; they must not be deleted, relabeled, or treated as invalid because no run was executed against them.

SECTION 7: OUTCOME LABEL SET AND DEFINITIONS [LOCKED]

7.1 Locked outcome label set

Only the following outcome labels are permitted for intake runs:

- Observed as asserted
- Not observed as asserted
- Constrained
- Insufficiently specified for bounded execution
  No additional labels, sublabels, qualifiers, or paraphrased variants are permitted.

7.2 Observed as asserted

Meaning: Under bounded execution parameters for that RUN_UNIT and browser context, the asserted website condition manifested.

7.3 Not observed as asserted

Meaning: Under bounded execution parameters for that RUN_UNIT and browser context, the asserted website condition did not manifest.

7.4 Constrained

Meaning: Execution was blocked by an identifiable technical constraint that prevented a bounded attempt of the asserted condition, and the constraint maps to one of the locked constraint_class values defined in this doctrine.

7.5 Insufficiently specified for bounded execution

Meaning: The submitted materials do not provide sufficient bounded specification to attempt the asserted condition under controlled parameters for that RUN_UNIT and browser context.

7.6 Locked hard constraints on labels

Do not:

- replace outcome labels
- collapse outcome labels
- reorder outcome labels
- paraphrase outcome labels
- introduce aggregate labels
- introduce summary labels
- introduce multi-outcome statements for a single run

7.7 One run, one outcome

Each run must produce exactly one outcome label from this locked set.

7.8 Why this section is rigid

Outcome labels control sufficiency, stop logic, traceability, and downstream determination behavior. If labels drift, intake logic drifts with them.

SECTION 8: EXECUTION CONTEXT RIGOR [LOCKED]

8.1 Contexts are parameter locks, not suggestions

Replicated Desktop Browser Context and Replicated Mobile Browser Context are fixed emulation profiles. Desktop and Mobile are parameter locks, not descriptive labels. Operators must not resize live browser windows or dev tools panels to approximate these contexts. Contexts are instantiated only through explicit viewport and environment parameters in a controlled browser automation environment.

8.2 Replicated Desktop Browser Context, locked baseline

The locked Desktop baseline is:

- viewport: 1366 × 900 CSS pixels, fixed
- browser zoom: 100%
- deviceScaleFactor: 1
- orientation: landscape
- isMobile: false
- hasTouch: false
  Desktop baseline is a first-class intake baseline.

8.3 Replicated Mobile Browser Context, locked baseline

The locked Mobile baseline is:

- viewport: 393 × 852 CSS pixels, portrait only, fixed
- orientation: Portrait, locked
- browser zoom: 100%
- deviceScaleFactor: 1
- isMobile: true
- hasTouch: true
  Mobile baseline is a first-class intake baseline.

8.4 Peer-baseline scope rule

Desktop baseline and Mobile baseline are peer baselines for intake.

For generic website accessibility allegations, both Replicated Desktop Browser Context and Replicated Mobile Browser Context must be brought into scope under the intake determination logic.

Where the submitted materials expressly cabin the asserted condition to one baseline only, intake must issue the corresponding single-baseline determination.

Where one baseline is feasible and the peer baseline is blocked under controlled parameters, intake must issue the corresponding constrained-baseline determination.

Generic accessibility phrasing must not be treated as Desktop-only by default. For generic website accessibility allegations, both Replicated Desktop Browser Context and Replicated Mobile Browser Context must be brought into scope unless the submitted materials expressly cabin the asserted condition to one baseline only.

8.5 Reflow separation

Reflow testing under WCAG 1.4.10 is operationally separate from baseline intake capture. Baseline viewport parameters are not derived from reflow math.

8.6 Reflow primary method

The locked primary reflow method uses:

- viewport width: 320 CSS pixels
- viewport height: not fixed; determined by the browser
- zoom: 100%
- orientation: portrait
- deviceScaleFactor: 1
- isMobile: false
- hasTouch: false

8.7 Reflow supplemental method

The locked supplemental reflow method uses:

- viewport width: 1280 CSS pixels
- viewport height: not fixed; determined by the browser
- zoom: 400%
- effective width: 320 CSS pixels
- orientation: portrait
- deviceScaleFactor: 1
- isMobile: false
- hasTouch: false

8.8 Reflow does not alter baselines

Neither reflow method alters the locked Desktop or Mobile baselines defined above.

8.9 Why context rigor is strict

This section exists to prevent approximation. Intake contexts are locked parameter sets, not operator impressions of what Desktop or Mobile probably means.

SECTION 9: MECHANICAL NOTE DISCIPLINE AND CONSTRAINT CLASSIFICATION [LOCKED]

9.1 Purpose of notes

Notes exist solely to prevent drift and preserve mechanical clarity when an attempted run cannot be bounded or is blocked. Notes must not create narrative and must not introduce evaluative framing.

9.2 Note permission gate

Notes are permitted only when:

- the outcome label is Constrained (per-run note)
- the outcome label is Insufficiently specified for bounded execution (per-run note)
- the matter determination is DETERMINATION: ELIGIBLE FOR DESKTOP TECHNICAL RECORD BUILD / MOBILE BASELINE: CONSTRAINED (matter-level note)
- the matter determination is DETERMINATION: ELIGIBLE FOR MOBILE TECHNICAL RECORD BUILD / DESKTOP BASELINE: CONSTRAINED (matter-level note) Gates 1 and 2 authorize a single mechanical note attached to the affected run record. Gates 3 and 4 authorize a single mechanical note attached to the matter-level determination record.

9.3 Notes are prohibited for

- Observed as asserted
- Not observed as asserted
- any statement about sufficiency reached or not reached
- any statement about run ordering, context ordering, or interleaving decisions
- any attempt to explain, defend, or argue outcome selection

9.4 Mechanical sentence rule

Any permitted note must be exactly one mechanical sentence that states the blocking condition only.

9.5 Content restrictions for notes

Permitted notes must contain:

- no legal terms
- no evaluative language
- no blame language
- no speculation
- no motive, severity, or probability framing

9.6 Locked constraint_class enum set

If the outcome label is Constrained, the internal record must use exactly one of the following constraint_class values only:

- AUTHWALL
- BOTMITIGATION
- GEOBLOCK
- HARDCRASH
- NAVIMPEDIMENT
  Operators must not create new constraint_class values.

9.7 Unmapped blocking condition rule

If a blocking condition does not map to one of the above values, the run must not be labeled Constrained. Classify the run as Insufficiently specified for bounded execution and use the mechanical note to state the missing bounded path or trigger.

9.8 Why notes are restricted

When a note is permitted, its internal classification must use exactly one of the following locked values:

- outcome_constrained
- outcome_insufficiently_specified
- determination_desktop_eligible_mobile_constrained
- determination_mobile_eligible_desktop_constrained
  No additional note classification values are permitted.

SECTION 10: RUN SEQUENCING, STATE ISOLATION, AND TEMPORAL LOGGING [LOCKED]

10.1 Sequencing rule

When both Desktop and Mobile contexts are in scope for the matter, intake runs must alternate contexts: Desktop, then Mobile, then Desktop, then Mobile, continuing until a stop condition is reached. When only one baseline is in scope for the matter, all intake runs are executed in that baseline context and no interleaving occurs.

When both baselines are in scope but one baseline is blocked under controlled parameters, runs in the constrained baseline must still be attempted in their alternating slots. Each such attempt must produce a

distinct run with an applicable Constrained outcome and constraint_class, and consumes a run slot. The alternating sequence does not skip or collapse because one baseline is constrained.

10.2 Clean-state isolation between every run

Between every run, the prior execution state must not carry forward.

10.3 Locked clean-state requirements

Each run must execute in a fresh, isolated browser context. Storage state must not persist between runs. Any attempt that cannot be performed under clean-state isolation must not proceed to condition evaluation. The blocking condition must be labeled using the locked outcome labels and, when applicable, a locked constraint_class value, and the isolation failure must be preserved as internal execution metadata for that run. Clean-state isolation applies identically to Desktop and Mobile runs.

10.4 Temporal logging

Each run must record internal timestamps sufficient to show sequencing and separation, at minimum:

- run_start time
- run_end time

10.5 Temporal logging is internal only

- These timestamps are internal execution metadata only. They are not exposed in any external-facing intake output and must not be referenced indirectly, for example by describing runs as lengthy, brief, or extensive.

10.6 Why this section exists

This section prevents sequencing manipulation, state carry-forward, and time-based rhetorical leakage. Intake must remain mechanically sequenced, clean-state isolated, and externally nondisclosive.